

2차시험 치팅페이어

시험 직전 빠른 복습용으로, 문제 prompt와 답안만 이어 붙인 파일입니다.

양인순교수 기출확정

1. Markov Decision Process (MDP)

MDP를 구성하는 핵심 요소 5가지를 쓰고, 각 요소를 설명하시오.

answer

MDP는 상태(state), 행동(action), 전이확률(transition probability), 보상함수(reward function), 할인율(discount factor)로 구성된다.

- 상태(state): agent가 처한 상황
- 행동(action): agent가 선택할 수 있는 의사결정
- 전이확률(transition probability): 행동 후 다음 상태가 어떻게 정해지는지
- 보상함수(reward function): 행동의 좋고 나쁨
- 할인율(discount factor): 미래 보상을 현재보다 얼마나 덜 중요하게 볼지를 나타낸다

2. Value Function / Q-Function

Value function과 Q-function의 차이를 설명하시오. 특히 Q-function을 알면 policy를 어떻게 얻을 수 있는지 설명하시오.

answer

Value function은 어떤 상태(state)의 장기적 가치(long-term value)를 나타내고, Q-function은 어떤 상태에서 특정 행동(action)을 선택했을 때의 장기적 가치를 나타낸다.

Q-function을 알면 각 상태에서 Q 값이 가장 큰 행동을 선택하면 되므로 policy를 얻을 수 있다.

3. Q-Learning

Q-learning은 환경 모델(environment model)을 정확히 알지 못해도 학습할 수 있다. 그 이유를 벨만 방정식(Bellman equation)을 "직접 푸는 것"과 비교하여 설명하시오.

answer

Bellman equation을 직접 풀려면 전이확률(transition probability)과 보상함수(reward function) 같은 모델 정보(model information)가 필요하다.

반면 Q-learning은 실제로 얻은 경험, 즉 상태(state), 행동(action), 다음 상태(next state), 보상(reward) 데이터를 사용해 Q 값을 점진적으로 갱신한다. 따라서 환경 모델을 명시적으로 알지 않아도 경험을 통해 optimal Q-function에 가까워지도록 학습할 수 있다.

4. Deep Q-Network

DQN에서 사용하는 experience replay와 target network의 역할을 각각 설명하시오.

answer

Experience replay는 transition 데이터를 replay buffer에 저장한 뒤 무작위로 샘플링(random sampling)하여 학습함으로써 순차적 샘플 사이의 상관성(correlation)을 줄인다.

Target network는 target 값을 계산하는 네트워크를 일정 기간 고정하거나 천천히 업데이트하여, 고정된 target을 둔 회귀 문제(regression problem)처럼 학습하게 해 안정성(stability)을 높인다.

5. Double Q-Learning

DQN에 Double Q-learning trick을 추가하는 주된 원인과, 구체적으로 어떻게 활용되는지 말하시오.

answer

Double DQN의 목적은 Q 값의 과대평가(overestimation)를 줄이는 것이다. 일반 DQN에서는 같은 네트워크가 행동 선택(action selection)과 가치 추정(value estimation)을 함께 담당하면서 max 연산으로 인해 값이 과대평가될 수 있다.

Double DQN은 현재 Q-network를 행동 선택에 사용하고 target network를 가치 추정에 사용하여 이 문제를 완화한다.

6. Policy Gradient

Policy gradient method가 value-based method와 다른 점을 설명하시오.

answer

Value-based method는 좋은 행동을 고르기 위해 value function 또는 Q-function을 먼저 학습한다. 반면 policy gradient method는 policy 자체를 파라미터화(parameterization)하고, 기대 보상(expected return)이 증가하는 방향으로 policy parameter를 직접 업데이트한다.

즉, "행동의 가치 평가 함수"를 중심으로 학습하는 것이 value-based method이고, "행동 선택 규칙 자체"를 직접 최적화하는 것이 policy gradient method이다.

7. Actor-Critic

Actor-Critic 방법에서 actor와 critic의 역할을 각각 설명하시오.

answer

Actor는 policy를 나타내며 어떤 행동을 선택할지 결정한다. Critic은 actor가 따르는 policy 또는 선택한 행동의 가치를 평가하여 actor 업데이트에 필요한 신호를 제공한다.

8. DDPG

DDPG가 DQN보다 continuous action space에 더 적합한 이유를 설명하시오.

answer

DQN은 각 행동의 Q 값을 비교하여 가장 좋은 행동을 고르는 방식이므로, 행동 공간(action space)이 이산적(discrete)일 때 자연스럽다. 행동이 연속적(continuous)이면 가능한 행동이 무한히 많아 모든 행동의 Q 값을 비교하기 어렵다.

DDPG는 deterministic actor가 연속 행동(continuous action)을 직접 출력하고, critic이 그 행동의 가치를 평가하므로 continuous control에 적합하다.

9. TRPO / PPO

TRPO와 PPO의 개념과 차이를 서술하시오.

answer

TRPO는 Policy가 너무 크게 변하지 않도록 KL Divergence Constraint를 두고 이를 풀기 위해 헤시한 같은 2차 정보를 사용한다. 그래서 이 과정은 계산적으로 무겁다.

PPO는 이런 복잡한 constrained optimization 대신 클리핑되는 오프젝티브를 사용하여 Policy변화가 지나치게 커지는 것을 제한한다.

따라서, PPO는 TRPO의 안정성 아이디어를 더 단순한 1차 최적화 방식으로 구현한 방법이다.

10. Model-Based RL / Model-Free RL

Model-based RL과 Model-free RL의 개념과 차이를 서술하시오.

answer

Model Free RL은 전이 확률이나 보상함수를 명시적으로 학습하지 않고 policy나 value function을 직접 학습한다.

Model-based RL은 전이 모델과 보상 모델을 학습하고, 그 모델을 이용해 폴리스를 업데이트 한다.

Model Free RL은 환경 구조를 명시적으로 활용하지 않기 때문에, 많은 샘플이 필요하고 학습이 느릴 수 있다.

김찬우교수 기출확정

1. Shallow Fusion

Domain-Specific Language Model(LM)을 가지고 있다고 하자. 이 Domain Specific LM을 이용하여, 이 Domain에서의 음성 인식 성능을 더 높이는 방법에 Shallow-Fusion이라는 방법이 있는데, 이 방법이 어떻게 동작하는 것인지 설명하라.

answer

Shallow Fusion은 음성 인식 모델이 예측한 토큰 분포의 로그 확률과 별도로 학습된 Language Model(LM)이 예측한 토큰 분포의 로그 확률을 Linear Interpolation 해준다. 이 결합된 로그 확률을 가장 크게 만드는 토큰을 선택한다.

2. Wav2Vec 2.0

(a)

Wav2Vec 2.0에서 pre-training을 할 때 BERT의 경우와는 달리 음성은 바로 discrete token으로 되어 있지 않기 때문에 Masked Language Model(MLM) 방식의 Cross-Entropy(CE) Loss를 바로 사용할 수는 없다. 이 문제를 해결하기 위해서 Wav2Vec 2.0에서는 내부적으로 Vector Quantizer의 codebook을 함께 훈련하는데, 이를 위해서 Wav2Vec 2.0에서는 어떠한 loss 두 개를 사용하는지 설명하라.

(b)

Codebook에서 Codevector를 선택할 때 특정 Codevector들만 자주 선택되면 잘 설계된 Codebook이 아니다. 이를 위해서 Wav2Vec 2.0에서는 어떠한 loss를 사용하였는지 ((a)에 답한 두 개 loss 중의 하나이다) 그리고 이 loss와 Codevector를 선택하는 확률의 Entropy와의 관계는 어떤지 설명하라.

answer

(a)

Contrastive Loss와 Diversity Loss가 함께 사용된다.

(b)

Diversity Loss가 사용되며 이 Loss는 Codevector들이 선택되는 평균 확률의 Entropy의 마이너스 값이다. Diversity Loss가 작으려면 Entropy가 커져야 하므로 Codevector를 선택하는 확률을 Uniform Distribution에 가깝게 만들게 된다.

김희승교수 기출확정

1. Day 1 — Mel-spectrogram

대부분의 음성 AI 모델은 raw 파형을 직접 사용하지 않고 mel-spectrogram을 입력으로 사용한다. mel-spectrogram을 쓰면 raw 파형 대비 어떤 점이 유리한지 1가지 이상 설명하시오.

answer

mel-spectrogram을 쓰는 이유는 다음 중 어느 하나만 답해도 정답이다.

1. 차원/시퀀스 길이 축소: raw 파형은 1초당 16,000~22,050 샘플로 매우 길지만, mel-spec은 1초당 약 80~100 프레임으로 100배 이상 짧다. 모델이 다루기 훨씬 쉽다.
2. 인간 청각 특성 반영: 인간 귀는 저주파에 더 민감하고 고주파에 둔감한데, mel 필터뱅크가 이 비선형성을 반영한다. 따라서 모델이 인간이 듣는 방식으로 음성을 본다.
3. phase 정보 제거 → 학습 용이: raw 파형의 phase는 perceptually 거의 무의미한데 학습을 방해한다. mel은 magnitude만 쓰므로 학습이 안정적이다.

1, 2, 3 중 하나만 정확히 답하면 만점.

2. Day 3 — TTS의 두 단계

현대 TTS 시스템은 보통 Acoustic Model과 Vocoder 두 모듈로 나뉜다. 각 모듈이 무엇을 입력으로 받고 무엇을 출력하는지 간단히 설명하시오.

answer

모듈	입력	출력	예시
Acoustic Model	텍스트 또는 phoneme	mel-spectrogram	Tacotron 2, FastSpeech 2
Vocoder	mel-spectrogram	raw 파형(waveform)	WaveNet, HiFi-GAN

전체 흐름:

text → (Acoustic Model) → mel → (Vocoder) → waveform

3. Day 1 — 음성 task의 공통 문제

ASR(음성 인식)과 TTS(음성 합성)는 표면적으로는 다른 일을 하지만, 두 task 모두 공통적으로 풀어야 하는 핵심 문제가 있다. 이 문제의 이름을 쓰고, 왜 음성 task에서 이 문제가 발생하는지 한 줄로 설명하시오.

answer

문제 이름: Alignment (정렬) 문제

발생 이유: 텍스트(글자 단위)와 음성(시간 단위, 프레임 단위)은 길이가 서로 다르고, 음성이 훨씬 길며, 고정된 비율이 아닌 가변적인 길이 비율을 갖기 때문이다. 또는 사람이 같은 말이라도 어떤 속도로 말하고 어떤 환경에서 말하는지 등의 상황이 다르기 때문이다.

즉, “어느 글자가 음성의 어느 구간에 해당하는지”를 모델이 알아내야 한다.

ASR/TTS 모두 이 alignment를 어떻게 푸느냐가 핵심 설계 포인트.

4. Day 3 — Tacotron 2 vs FastSpeech 2

Tacotron 2와 FastSpeech 2는 둘 다 텍스트로부터 mel-spectrogram을 생성한다. 그런데 추론(inference) 속도에서 FastSpeech 2가 훨씬 빠르다. 이 속도 차이가 왜 발생하는지 설명하시오.

answer

모델	방식	추론 방식	속도 특성
Tacotron 2	AR (autoregressive, 자기회귀적)	mel 프레임을 한 프레임씩 순차적으로 생성한다. t번째 프레임은 t-1번째 프레임을 보고 만든다.	길이가 T만큼 순차 forward pass가 필요하다.
FastSpeech 2	Non-AR (non-autoregressive, 비자기회귀적)	모든 mel 프레임을 병렬로 한 번에 생성한다.	1회 forward pass면 끝난다.

핵심은 순차(serial) vs 병렬(parallel) 추론이다. 같은 GPU에서도 FastSpeech 2는 GPU 병렬성을 충분히 활용 가능하므로 수십~수백 배 빠르다.

황승원교수 기출확정

1. Edit Distance

Given two strings "FAST" and "CATS", calculate the minimum edit distance using:

- Insertion cost: 1
- Deletion cost: 1
- Substitution cost: 2

Explain your answer and show the alignment.

두 스트링 "FAST"와 "CATS" 사이의 edit distance를 구하라. 단, 삽입 및 삭제 연산의 비용은 각각 1로, 대체 연산의 비용은 2로 둔다. 둘 사이의 alignment도 함께 제시하라.

answer

편집 거리는 4이다. 가능한 경렬은 여러 가지가 있다.

정렬 1

```
FAST*  
CA*TS
```

적용된 연산:

1. F를 C로 대체한다. 비용 2
2. A는 그대로 둔다. 비용 0
3. S를 삭제한다. 비용 1
4. T는 그대로 둔다. 비용 0
5. 마지막에 S를 삽입한다. 비용 1

총 비용: $2 + 1 + 1 = 4$

정렬 2

```
*FAST*  
C*AT*TS
```

적용된 연산:

1. 맨 앞에 C를 삽입한다. 비용 1
2. F를 삭제한다. 비용 1
3. A는 그대로 둔다. 비용 0
4. S를 삭제한다. 비용 1
5. T는 그대로 둔다. 비용 0
6. 마지막에 S를 삽입한다. 비용 1

총 비용: $1 + 1 + 1 + 1 = 4$

2. Byte-Pair Encoding (BPE)

Given the corpus below, perform tokenization based on byte-pair encoding (BPE).

Corpus: new new new newer newest renew renewed renewing

Initial vocabulary: { _, d, e, g, i, n, r, s, t, w, l, y }

Note: For this question, if multiple pairs have the same highest frequency, resolve the tie by choosing the pair that comes first in alphabetical order. You can determine the alphabetical order by the first element of the pairs if they are not identical. For example, (a, z) would be chosen over (b, a). We assume that the token '_' comes before 'a' in the alphabetical order.

(1) After adding end-of-word tokens () and segmenting the corpus into individual characters, fill in the frequency table below for the initial tokenized corpus.

Token sequence	Frequency
--- ---	1
n/e/w/_	1
n/e/w/e/r/_	1
n/e/w/e/s/t/_	1
r/e/n/e/w/_	1
r/e/n/e/w/e/d/_	1
r/e/n/e/w/i/n/g/_	1

(2) What are the first THREE merges that the BPE algorithm will perform? List them in order, showing:

- The pair being merged
- The frequency of that pair
- The new token created

(3) Show the state of the corpus after these three merges have been applied.

- (4) After performing 5 merges total, the vocabulary includes: {_, d, e, g, i, n, r, s, t, w, l, y, ew, re, new, new_, newe}. Tokenize the new word "newly" using this vocabulary.
- (5) This BPE-level tokenization has been proposed due to this problem in word-level tokenization. Explain concisely what the problem is.

answer

(1) 초기 빈도표

```
| 토큰 sequence | 빈도 |
|---|---|
| n/e/w/_ | 3 |
| n/e/w/e/r/_ | 1 |
| n/e/w/e/s/t/_ | 1 |
| r/e/n/e/w/_ | 1 |
| r/e/n/e/w/e/d/_ | 1 |
| r/e/n/e/w/i/n/g/_ | 1 |
```

(2) 처음 세 번의 병합

- 첫 번째 병합: $e/w \rightarrow ew$, 빈도 8
- 두 번째 병합: $n/ew \rightarrow new$, 빈도 8. 첫 번째 병합을 적용한 뒤의 빈도이다.
- 세 번째 병합: $new/_ \rightarrow new_$, 빈도 4

(3) 세 번 병합한 뒤의 corpus 상태

- $new_ : 3$
- $new/e/r/_ : 1$
- $new/e/s/t/_ : 1$
- $r/e/new_ : 1$
- $r/e/new/e/d/_ : 1$
- $r/e/new/i/n/g/_ : 1$

(4) "newly" 토큰화

- 시작 상태: $n/e/w/l/y/_$
- 앞에서부터 가장 긴 매칭을 찾으면 vocabulary 안에 "new"가 있다.
- "new"를 적용하면 $new/l/y/_$ 가 된다.
- 다음 위치에서는 "l", "ly", "ly_" 중 추가로 합칠 수 있는 토큰이 없다.
- 결과: $new/l/y/_$

(5) 단어 단위 토큰화는 학습 때 보지 못한 단어가 그대로 out-of-vocabulary 문제가 된다. BPE 같은 subword 토큰화는 보지 못한 단어도 이미 학습된 더 작은 조각들의 조합으로 표현할 수 있어 희귀어와 신조어를 처리하기 쉽다.

3. N-gram Language Model

Suppose you are given the following small corpus of three sentences:

- $\langle s \rangle$ i like nlp $\langle /s \rangle$
- $\langle s \rangle$ i like math $\langle /s \rangle$
- $\langle s \rangle$ i write code $\langle /s \rangle$

Based on this corpus, calculate the following probabilities using the Maximum Likelihood Estimation (MLE) method. Show your work. Be sure to count $\langle s \rangle$ and $\langle /s \rangle$ as valid tokens.

- What is the unigram probability of the word "i"?
- What are the following bigram probabilities?
 - $P(\text{like} | i)$
 - $P(\text{nlp} | \text{like})$
 - $P(\text{code} | \text{like})$
- What is the probability of the sentence " $\langle s \rangle$ i like math $\langle /s \rangle$ " according to the bigram model?
- Explain briefly why this count-based LM fails and how CBOW improves upon it.
- RNN-based language models handle sequences. Explain how RNN/LSTM addresses the problem of CBOW.
- Describe one major limitation of RNN/LSTM solved by Transformer-based models.

answer

먼저 corpus에서 unigram과 bigram count를 정리한다.

Unigram count

- $\langle s \rangle$: 3
- i: 3
- like: 2
- nlp: 1
- $\langle /s \rangle$: 3
- math: 1
- write: 1
- code: 1

- 전체 토큰 수: 15

Bigram count

- (<s>, i): 3
- (i, like): 2
- (i, write): 1
- (like, nlp): 1
- (like, math): 1
- (nlp, </s>): 1
- (math, </s>): 1
- (write, code): 1
- (code, </s>): 1

(1) Unigram 확률

unigram 확률은 해당 단어의 등장 횟수를 corpus 전체 토큰 수로 나누어 계산한다.

$$P(\text{"i"}) = \frac{\text{count}(\text{"i"})}{\text{전체 토큰 수}} = \frac{3}{15} = \frac{1}{5}$$

(2) Bigram 확률

bigram의 최대우도추정은 다음 식을 사용한다: $P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$

- $P(\text{like} \mid \text{i}) = \frac{C(\text{i, like})}{C(\text{i})} = \frac{2}{3}$
- $P(\text{nlp} \mid \text{like}) = \frac{C(\text{like, nlp})}{C(\text{like})} = \frac{1}{2}$
- $P(\text{code} \mid \text{like}) = \frac{C(\text{like, code})}{C(\text{like})} = \frac{0}{2} = 0$

(3) Bigram model에서 문장 확률

문장 확률은 문장을 구성하는 각 bigram 조건부 확률의 곱으로 계산한다.

$$P(\text{<s> i like math </s>}) = P(\text{i} \mid \text{<s>}) \times P(\text{like} \mid \text{i}) \times P(\text{math} \mid \text{like}) \times P(\text{</s>} \mid \text{math})$$

- $P(\text{i} \mid \text{<s>}) = \frac{C(\text{<s>, i})}{C(\text{<s>})} = \frac{3}{3} = 1$
- $P(\text{like} \mid \text{i}) = \frac{C(\text{i, like})}{C(\text{i})} = \frac{2}{3}$
- $P(\text{math} \mid \text{like}) = \frac{C(\text{like, math})}{C(\text{like})} = \frac{1}{2}$
- $P(\text{</s>} \mid \text{math}) = \frac{C(\text{math, </s>})}{C(\text{math})} = \frac{1}{1} = 1$

$$P(\text{<s> i like math </s>}) = 1 \times \frac{2}{3} \times \frac{1}{2} \times 1 = \frac{2}{6} = \frac{1}{3}$$

(4) Count-based LM은 buy와 purchase처럼 의미가 비슷한 단어 사이에서도 정보를 공유하지 못한다. CBOW는 주변 context로부터 중심 단어를 예측하면서 embedding을 학습하므로, 비슷한 context에 등장하는 단어들이 유사한 벡터를 갖게 되어 정보 공유가 가능하다.

(5) CBOW는 주변 단어를 bag처럼 취급하여 단어 순서 정보를 충분히 반영하지 못한다. RNN/LSTM은 단어를 순차적으로 입력받으며 hidden state를 갱신하므로 단어 순서와 앞선 context를 반영할 수 있다.

(6) 병렬화: RNN/LSTM은 이전 hidden state에 의존하여 순차적으로 계산해야 하므로 병렬화가 어렵다. Transformer는 attention을 통해 모든 토큰 사이의 관계를 한 번에 계산할 수 있어 병렬화가 쉽고 긴 거리 의존성도 더 직접적으로 다룰 수 있다.

4. Document Retrieval (BoW, TF-IDF)

You are designing a document retrieval system that must rank three short documents for a user's query. Answer the following questions:

(1) In classic Bag-of-Words (BoW), how is a document represented, and what key limitation of BoW motivates moving beyond raw counts? (Answer in one sentence, assume the vocabulary size of |V|.)

(2) Given TF-IDF for this corpus, let the user query be q: "cats sleep". Form q's tf-idf vector (in this case, assume count of word is tf-idf for query), and compute cosine(q, D₁), cosine(q, D₂), cosine(q, D₃), and rank the documents. Assume the following matrix represents the TF-IDF weighted document vectors for the corpus.

Term	cats	dogs	sleep	mat	sofa	chase
D ₁	1	1	0	1	1	3
D ₂	1	0	1	1	1	0
D ₃	1	1	1	0	0	3

(3) Explain in one sentence why cosine similarity is preferred over Euclidean (L2) distance for ranking query-document similarity.

(4) State one reason to move from sparse BoW/TF-IDF vectors to dense word embeddings for retrieval (e.g., semantic similarity), and name one common way embeddings are learned.

answer

(1) 문서는 vocabulary 크기인 |V|차원의 count vector로 표현된다. 각 차원은 하나의 term에 대응한다. BoW는 단어 순서를 무시하므로, 의미와 관련성 판단에 중요한 구·위치·문맥 정보를 잃는다는 한계가 있다.

(2) q ("cats sleep"): [1, 0, 1, 0, 0, 0].

$$\cos(q, D_1) = \frac{1 + 1}{\sqrt{2} \cdot \sqrt{11}}$$

$$\cos(q, D_2) = \frac{1}{\sqrt{2} \cdot \sqrt{11}}$$

$$\cos(q, D_3) = \frac{1}{\sqrt{2} \cdot \sqrt{11}}$$

순위: D₁ > D₂ = D₃

(3) Euclidean distance는 vector 길이의 영향을 크게 받기 때문에 긴 문서가 불리해질 수 있다. 반면 cosine similarity는 vector의 방향, 즉 term 비율의 유사성을 비교하므로 query-document 유사도 랭킹에 더 적합하다.

(4) Dense embedding은 정확히 같은 단어가 겹치지 않아도 의미적으로 가까운 단어와 구를 비슷한 벡터 공간에 배치할 수 있어 semantic similarity를 반영한다. 대표적인 학습 방식으로는 local context window를 이용하는 skip-gram이나 CBOW가 있다.

5. Multi-class Classification Metrics

You have built a classifier to categorize news articles into three classes: 'Sports', 'Business', and 'Tech'. After running the classifier on a test set, you get the following confusion matrix:

```
| | Predicted 'Sports' | Predicted 'Business' | Predicted 'Tech' |
|---|---|---|---|
| True 'Sports' | 40 | 8 | 2 |
| True 'Business' | 5 | 80 | 15 |
| True 'Tech' | 5 | 12 | 83 |
```

- (1) Calculate the Precision, Recall, and F1-score for each class.
- (2) Calculate the macro-averaged F1-score.
- (3) Calculate the micro-averaged F1-score.

answer

- (1) Precision, Recall, F1

macro average를 구하기 위해 먼저 각 class별 Precision, Recall, F1을 계산한다.

Sports class

- $TP = 40$
- $FP = 5 + 5 = 10$
- $FN = 8 + 2 = 10$
- $Precision_S = TP / (TP + FP) = 40 / 50 = 0.80$
- $Recall_S = TP / (TP + FN) = 40 / 50 = 0.80$
- $F1_S = 2 \times (P \times R) / (P + R) = 2 \times (0.80 \times 0.80) / (0.80 + 0.80) = 0.80$

Business class

- $TP = 80$
- $FP = 8 + 12 = 20$
- $FN = 5 + 15 = 20$
- $Precision_B = 80 / 100 = 0.80$
- $Recall_B = 80 / 100 = 0.80$
- $F1_B = 2 \times (0.80 \times 0.80) / (0.80 + 0.80) = 0.80$

Tech class

- $TP = 83$
- $FP = 2 + 15 = 17$
- $FN = 5 + 12 = 17$
- $Precision_T = 83 / 100 = 0.83$
- $Recall_T = 83 / 100 = 0.83$
- $F1_T = 2 \times (0.83 \times 0.83) / (0.83 + 0.83) = 0.83$

- (2) Macro average

- $Macro-Precision = (0.80 + 0.80 + 0.83) / 3 = 2.43 / 3 = 0.81$
- $Macro-Recall = (0.80 + 0.80 + 0.83) / 3 = 2.43 / 3 = 0.81$
- $Macro-F1 = (0.80 + 0.80 + 0.83) / 3 = 2.43 / 3 = 0.81$

- (3) Micro average

micro average는 모든 class의 TP, FP, FN을 합산한 뒤 전체 기준으로 Precision, Recall, F1을 계산한다.

- $Total\ TP = 40 + 80 + 83 = 203$
- $Total\ FP = 10 + 20 + 17 = 47$
- $Total\ FN = 10 + 20 + 17 = 47$
- $Micro-Precision = 203 / (203 + 47) = 203 / 250 = 0.812$
- $Micro-Recall = 203 / (203 + 47) = 203 / 250 = 0.812$
- $Micro-F1 = 2 \times (Micro-P \times Micro-R) / (Micro-P + Micro-R) = 0.812$

6. Interpolation vs. Backoff

Explain the difference between interpolation and backoff in n-gram language models. Why are these techniques necessary?

N-gram 언어모델에서 interpolation과 backoff의 차이를 설명하라. 또한 이러한 기법들이 왜 필요한지 설명하라.

answer

(a) Interpolation은 trigram, bigram, unigram 등의 여러 n-gram 확률을 가중합하여 함께 사용하는 방식이다. 예컨대 bigram과 unigram 확률을 보간하면:

$$p(y_t \mid y_{<t}) = \lambda \cdot p_b(y_t \mid y_{t-1}) + (1 - \lambda) \cdot p_u(y_t)$$

(b) Backoff는 높은 차수의 n-gram이 존재하지 않거나 신뢰하기 어려울 때 더 낮은 차수의 n-gram으로 물러나서 확률을 추정하는 방식이다.

(c) 이러한 기법들은 data sparsity 문제를 완화하기 위해 필요하다. 즉, 학습 데이터에서 관측되지 않은 n-gram의 확률이 0이 되는 문제를 줄이고, 제한된 corpus에서도 더 안정적인 확률 추정을 가능하게 한다.

7. Naive Bayes Classification

In Naive Bayes Classification for spam filtering, explain (a) how Bayes' theorem is utilized, and (b) what assumption is adopted.

Spam filtering을 위한 Naive Bayes Classification에서 (a) Bayes 정리가 어떻게 활용되는지, 그리고 (b) 어떠한 assumption을 채택하고 있는지 설명하시오.

answer

(a) class c 와 document x 에 대해서 classification 문제는 아래와 같이 정의된다.

$$\hat{c} = \arg\max_{c \in C} P(c \mid \mathbf{x})$$

조건부 확률 $P(c \mid \mathbf{x})$ 를 직접 계산하기 위해 Bayes 정리를 적용하면 다음과 같다.

$$P(c \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid c) P(c)}{P(\mathbf{x})}$$

(b) Train set에서 $P(\mathbf{x} \mid c)$ 를 직접 구하기는 어렵다. 따라서 Naive Bayes classification에서는 document 안의 단어들이 class가 주어졌을 때 서로 조건부 독립이라고 가정하고, 아래처럼 곱으로 분해한다.

$$P(x_1, x_2, \dots, x_n \mid c) = \prod_{i=1}^n P(x_i \mid c)$$

8. POS Tagging

What are the two primary sources of linguistic information for POS tagging? Use the sentence "Bill saw that man yesterday" as your running example.

품사 태깅에 주요하게 활용되는 두 가지 정보는 무엇인가? 예시 문장 "Bill saw that man yesterday"를 활용하여 설명하라.

answer

(1) 단어 자체의 품사 확률 정보: 주어진 단어가 주로 어떤 품사로 활용되는지에 대한 정보이다. 예를 들어 'man'은 동사로 쓰이는 경우가 드물고 명사로 쓰이는 경우가 많다.

(2) 주변 단어와 태그의 문맥 정보: 주변 단어들의 품사 태그 패턴이 얼마나 자연스러운지에 대한 정보이다. 예를 들어 "Bill saw that man yesterday"에서 주변 단어들을 함께 고려하면 saw는 명사 NN보다 과거형 동사 VBD로 보는 것이 자연스럽다.

9. Dense Retrieval (DPR, ColBERT)

Answer the following questions regarding dense retrieval.

(1) Explain the in-batch negatives strategy in DPR training and why it is efficient.

DPR 학습 과정에서의 in-batch negative 전략이 무엇인지 설명하고 효율성 측면에서의 장점을 논하시오.

(2) Explain how ColBERT works, and discuss how it balances the strengths and weaknesses of cross- and bi-encoders.

ColBERT가 어떻게 동작하는지 서술하고, cross- 및 bi-encoder 검색의 장단점 사이에서 어떤 균형을 제공했는지 설명하라.

answer

(1)

(a) In-batch negative란 같은 minibatch 안에 있는 쿼리-문서 (q, d) 쌍들이 주어졌을 때, q_i 의 positive 문서가 아닌 다른 문서 d_j ($i \neq j$)들을 negative 문서로 사용하는 전략이다.

(b) Negative 문서를 별도로 샘플링하는 과정을 생략할 수 있고, minibatch 안에서 이미 계산된 문서 embedding을 재사용하므로 각 쿼리마다 별도 negative 문서를 다시 encoding하는 비용도 줄어든다.

(2)

(a) ColBERT는 쿼리와 문서를 각각 encoding하여 각 토큰의 embedding을 얻은 뒤, 쿼리 토큰과 문서 토큰 사이의 late interaction으로 점수를 계산한다.

(b) 쿼리와 문서를 각각 하나의 vector embedding으로 encoding하는 bi-encoder와 비교하면, ColBERT는 토큰 단위 정보를 더 많이 보존하고 쿼리-문서 토큰 사이의 interaction을 가능하게 한다.

(c) 쿼리와 문서 토큰을 하나의 입력 시퀀스로 이어 붙여 encoding하는 cross-encoder와 비교하면, ColBERT는 문서 embedding을 offline에서 미리 계산해둘 수 있어 test time 비용을 줄인다. 즉, bi-encoder보다 표현력이 높고 cross-encoder보다 검색 비용이 낮은 절충점을 제공한다.

10. Parameter-Efficient Fine-Tuning

Answer the following questions regarding parameter-efficient fine-tuning.

(1) What is the main idea of adapter-based fine-tuning?

Adapter 기반의 미세 조정 훈련 기법의 핵심 아이디어는 무엇인가?

(2) What is the main idea of low-rank adaptation (LoRA) and how does it reduce the cost of fine-tuning?

LoRA의 핵심 아이디어는 무엇이고, 어떻게 미세 조정 훈련의 비용을 더욱 줄이는가?

answer

(1) 기반 모델의 parameter는 freeze하고, 학습 가능한 작은 모듈인 adapter를 추가한 뒤 그 adapter 부분만 업데이트하도록 학습한다. 전체 모델을 다시 학습하지 않기 때문에 task별 추가 학습 비용과 저장 비용을 줄일 수 있다.

(2)

(a) LoRA는 기존 weight 전체를 직접 업데이트하지 않고, 기존 linear layer의 변화량을 low-rank update $\Delta W = BA$ 로 표현한다. 즉 큰 행렬 하나를 학습하는 대신 작은 행렬 A 와 B 를 학습한다.

(b) 이 방식은 학습 가능한 parameter 수를 크게 줄이므로 fine-tuning 과정의 연산량과 메모리 요구량을 줄인다. 또한 base model weight는 유지하고 LoRA weight만 task별로 저장할 수 있어 parameter-efficient하다.

11. GRPO (Group Relative Policy Optimization)

Consider the training objective of GRPO (group relative policy optimization), which is given as:

$$\mathcal{J}_{\mathrm{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{r_i(\theta)}{r_i(\theta_{\mathrm{old}})}, 1 + \epsilon \right) \right) \right] - \beta D_{\mathrm{KL}}(\pi_{\theta} \parallel \pi_{\mathrm{ref}})$$

where

$$r_i(\theta) = \frac{\pi_{\theta}(o_i \mid q)}{\pi_{\theta_{\mathrm{old}}}(o_i \mid q)}, \quad A_i = \frac{r_i - \mu}{\sigma}, \quad \mu = \frac{1}{G} \sum_{j=1}^G r_j, \quad \sigma = \mathrm{std}(r_1, \dots, r_G).$$

(1) What is the benefit of using the relative advantage, compared to the estimated advantage in PPO (proximal policy optimization)?

PPO에서 critic network를 두고 advantage를 추정하여 사용하는 것 대비, GRPO의 상대적인 advantage는 어떤 장점이 있는가?

(2) What is the role of clipping and the KL divergence term?

Clipping 및 KL divergence 항의 기능은 무엇인가?

answer

(1) PPO처럼 별도의 critic network로 advantage를 추정하지 않고, 같은 질문에 대한 여러 응답들의 reward를 group 안에서 상대적으로 정규화하여 advantage를 만든다. 따라서 critic network를 유지하고 학습하는 부담이 사라지며, 긴 reasoning task처럼 critic 학습이 어렵거나 비용이 큰 상황에서 장점이 크다.

(2) Clipping은 probability ratio가 일정 범위를 벗어나지 못하게 하여 한 번에 너무 큰 policy update가 일어나는 것을 막는다. KL divergence 항은 현재 policy가 reference policy에서 지나치게 멀어지는 것을 penalize한다. 두 항 모두 policy가 급격히 변해 학습이 불안정해지는 것을 방지하는 역할을 한다.

김희승교수 예상문제

예상 1. Day 2 — WER (Word Error Rate)

ASR(음성 인식) 시스템의 성능을 평가하는 표준 메트릭은 WER이다. WER의 정의를 식으로 쓰고, 다음 경우의 WER을 계산하시오. 또한 한국어/일본어처럼 띄어쓰기 규칙이 모호한 언어에서 WER을 그대로 쓸 때의 문제와 그 대안 메트릭의 이름도 한 줄로 답하시오.

- Reference: "The cat sat on the mat" (6 단어)
- Hypothesis: "The bat sat the mat" (5 단어)

answer

$$\text{WER} = \frac{S + D + I}{N_{\text{Reference}}} \times 100\%$$

(S = Substitution, D = Deletion, I = Insertion, N = Reference 단어 수. 작을수록 좋음.)

비교: cat → bat (S = 1), on → (없음) (D = 1), I = 0.

$$\text{WER} = \frac{1 + 1 + 0}{6} \approx 33.3\%$$

한국어/일본어는 "아버지가 방에" vs "아버지 가방에"처럼 발음이 같아도 띄어쓰기만 다르다면 WER이 부풀려진다. 그래서 글자 단위로 거리를 재는 CER (Character Error Rate) 을 대안으로 사용한다.

예상 2. Day 2 — CTC의 3가지 핵심 키워드

CTC(Connectionist Temporal Classification)는 최초의 신경망 기반 End-to-End ASR 모델이며, 강사는 CTC를 Blank, Collapse, Marginalize 세 키워드로 설명했다. 각 키워드가 무엇을 의미하는지 간단히 설명하시오.

answer

키워드	의미
Blank ()	인접한 같은 알파벳이 collapse되어 사라지지 않게 막는 방지턱, 긴 음성 frame과 짧은 텍스트의 길이 차이를 메우는 채움 토큰, 발음 간 전환 구간 완충 역할.
Collapse (B)	인코더 출력의 연속된 같은 토큰을 하나로 합친 뒤 blank를 제거해 최종 텍스트로 만드는 규칙. 예: [h,h,e,l ,l,o] → hello.
Marginalize	학습 시 정답 텍스트 y로 collapse되는 모든 가능한 path의 확률을 합쳐서 정답 확률로 사용. Forward 알고리즘(dynamic programming)으로 효율 계산.
답안에 세 키워드 정의가 있으면 정답.

예상 3. Day 2 — RNN-Transducer가 CTC의 한계를 극복하는 방식

CTC와 RNN-Transducer는 모두 End-to-End ASR 모델이지만, RNN-T는 CTC의 본질적인 단점을 보완하기 위해 등장했다. CTC의 그 단점이 무엇이며, RNN-T가 어떤 모듈을 추가해 어떻게 극복했는지 설명하시오.

answer

CTC의 단점: Conditional Independence (조건부 독립) 가정. 각 시점의 출력을 음성만 보고 독립적으로 예측하므로 문맥을 고려하지 못한다. 예: "I want some" 다음에 "ice cream" 대신 "I scream"이 나오는 식. 그래서 보통 외부 LM을 추가 loss로 결합해 완화한다.
RNN-T는 CTC 구조(Encoder + Softmax)에 두 모듈을 추가한다.
모듈	역할
Prediction Network	지금까지 생성한 텍스트를 RNN으로 압축.
Joint Network	Encoder 출력(음성) + Prediction Network 출력(과거 텍스트)을 결합해 다음 토큰 예측.
이로써 RNN-T는 $P(y_t | x_{1:t}, y_{1:t-1})$ 형태로 과거 토큰에 의존해 생성하므로 외부 LM 없이도 문맥을 반영할 수 있다. (참고: RNN-T의 blank는 "현재 time frame은 끝났으니 다음 frame으로 이동"이라는 의미로, CTC의 blank와 역할이 다르다.)

예상 4. Day 2 — Whisper의 Hallucination

Whisper는 OpenAI가 발표한 99개 언어 ASR 모델로, AED(Attention-based Encoder-Decoder) 계열의 가장 성공적인 사례다. 그러나 잘 알려진 한계로 Hallucination 이 있다. Hallucination이 무엇인지, 왜 발생하는지, 그리고 이를 완화하기 위해 사용할 수 있는 방법 1가지 이상을 답하시오.

answer

Hallucination: 실제 음성에는 없는 내용을 모델이 스스로 소설처럼 생성하는 현상. 무음 구간이나 애매한 입력에서 특히 자주 발생.
원인: Whisper는 68만 시간 규모의 weak-supervised web caption으로 학습되었기 때문에, 학습 데이터에 섞인 방송 멘트/자막 패턴이 모델의 text prior에 강하게 박혀 있다. Autoregressive decoder가 그 prior를 따라가며 그럴듯한 문장을 만들어버리는 것이 hallucination의 근본 원인이다.
완화 방법 (1가지 이상):
• Initial Prompt: 고유명사/도메인 키워드를 prefix로 미리 제공. (예: 키오스크 메뉴 이름 → "백악관 김정은"처럼 hint를 주면 LLM 구조가 그것을 참고해 정정한다.)
• 디코딩 옵션 조절: temperature, compression_ratio_threshold, condition_on_previous_text 등을 조절하면 반복 루프와 환각 출력을 줄일 수 있다.
답안에 "환각/소설 작성" + "원인(text prior 또는 noisy caption 학습)" + "완화법 1개" 가 있으면 정답.

예상 5. Day 1 — STFT 파라미터와 Hann Window

음성 신호를 Mel spectrogram으로 변환하기 위해 STFT(Short-Time Fourier Transform)를 적용한다. n_fft = 1024, hop_length = 256 으로 설정했을 때, 두 윈도우의 중첩 비율(%) 과 Linear Spectrogram의 세로 차원이 각각 얼마인지 계산하고, 신호를 사각형으로 단순히 잘라낼 때 발생하는 Spectral Leakage 문제와 이를 방지하기 위해 흔히 쓰는 윈도우 함수의 동작 원리를 간단히 설명하시오.

answer

• 중첩 비율 = $(n_{\text{fft}} - \text{hop}) / n_{\text{fft}} = (1024 - 256) / 1024 = 75\%$.
• 세로 차원 = $n_{\text{fft}} / 2 + 1 = 1024 / 2 + 1 = 513$.
(이 513이 너무 커서 Mel filterbank로 80(TTS) 또는 128(ASR) 차원으로 압축한다.)
Spectral Leakage: 신호를 사각형으로 똑 자르면 끝점에서 진폭이 0으로 급변하고, 이 급변이 분석 결과에 원치 않는 고주파 성분으로 누설된다.
Hann (Hanning) Window: 양 끝을 0으로 부드럽게 눌러주는 종 모양 윈도우. 가운데는 그대로 두고 끝으로 갈수록 진폭이 점진적으로 0에 수렴하므로, 끝점 급변이 사라져 고주파 누설이 거의 없어진다. (양 끝 정보 손실은 75% overlap이 보완.)
답안에 "75%", "513", "Hann/Hanning window + 부드러운 절단" 이 있으면 정답.

예상 6. Day 3 — FastSpeech 2의 Alignment 해결 방식

Tacotron 2는 attention으로 텍스트와 mel 사이의 alignment를 직접 학습하지만 학습이 불안정하다는 단점이 있다. FastSpeech 2는 이를 다른 방식으로 해결했다. FastSpeech 2가 alignment 문제를 어떻게 푸는지, 학습 시와 추론 시의 처리 방식을 각각 설명하시오.

answer

FastSpeech 2는 attention 대신 외부 forced aligner를 활용해 alignment를 해결한다.

시점	처리 방식
학습 시	학습 전에 외부 Forced Aligner (MFA 등)에 (텍스트, 음성) 쌍을 넣어 각 phoneme의 duration label을 미리 추출. Length Regulator가 이 duration에 따라 phoneme의 hidden state를 정수배 복제해 mel 길이로 늘림 → 디코더가 mel을 병렬(non-AR)로 한 번에 생성.
추론 시	음성이 없어 forced aligner를 못 쓰므로, 학습 중 함께 학습한 Duration Predictor (작은 NN)가 phoneme별 duration을 예측. 그 예측값으로 Length Regulator가 동일하게 복제.
이 구조 덕분에 FastSpeech 2는 attention 불안정 문제(반복/누락/꺾임)를 피하고 Tacotron 2 대비 ~1000배 빠른 추론이 가능하다. (단, 외부 aligner가 CPU 전용이라 1000만 샘플 규모에서는 alignment 전처리에 몇 달이 걸릴 수 있다는 단점도 있음.)
답안에 "forced aligner로 duration 추출" + "학습/추론 처리 차이 (duration predictor)" 가 있으면 정답.

항승원교수 예상문제

예상 1. Normalization, Lemmatization vs Stemming, BPE

Answer the following questions about word normalization and tokenization.

(1) Explain why text normalization is needed. Give two examples of surface form variation.

텍스트 normalization이 필요한 이유를 설명하고, surface form variation 예시를 두 가지 들어라.

(2) Compare lemmatization and stemming in terms of method, output quality, and computational cost.

lemmatization과 stemming을 방법, 출력 품질, 계산 비용 측면에서 비교하라.

(3) Why is word-level tokenization weak for rare words or unseen words? How does BPE address this issue?

word-level tokenization이 희귀어/미등장 단어에 약한 이유는 무엇이며, BPE는 이를 어떻게 해결하는가?

(4) Suppose BPE has already learned the tokens {low, lower, est, er, newest, new, wide, r, _}. Tokenize "lower_" and "newer_" greedily using the longest available tokens.

위 vocabulary가 있을 때 longest-match 방식으로 "lower_", "newer_"를 토큰화하라.

answer

(1) 같은 의미가 여러 표면형으로 나타나면 모델이 이를 서로 다른 단어로 취급해 sparsity가 커진다. Normalization은 이런 표현을 통일해 학습과 검색을 안정화한다.

예:

- USA, U.S.A., United States
- running, runs, ran, run
- 대소문자 차이: Apple vs apple

(2)

항목	Lemmatization	Stemming
방법	사전 + 언어학적 지식 + 형태소/POS 분석	단순 rule 기반으로 접사 제거
출력	실제 표제어를 반환하는 경향	실제 단어가 아닐 수 있음
정밀도	높음	낮거나 거칠 수 있음
비용	상대적으로 비쌈	가벼움
예시	running → run, better → good 가능	running → runn, studies → studi 가능

(3) Word-level tokenization은 vocabulary에 없는 단어를 OOV로 처리한다. 희귀어, 신조어, 복합어가 나오면 의미 있는 내부 구조를 활용하지 못한다.

BPE는 자주 등장하는 문자/부분문자열 쌍을 반복적으로 merge해 subword vocabulary를 만든다. 따라서 처음 보는 단어도 이미 학습된 subword 조각의 조합으로 표현할 수 있다. 예를 들어 running, runner가 공통적으로 run 또는 runn 같은 조각을 공유할 수 있다.

(4)

- "lower_": longest token lower가 있으므로 lower / _
- "newer_": newest는 맞지 않고 new가 가장 긴 prefix. 이후 er, _가 있으므로 new / er / _

예상 2. Language Model Smoothing + Bayesian Classification

Consider a count-based language model and a Naive Bayes classifier.

(1) In an N-gram language model, explain why an unseen N-gram can make the whole sentence probability zero.

N-gram 언어모델에서 관측되지 않은 N-gram이 문장 전체 확률을 0으로 만드는 이유를 설명하라.

(2) Write the Add-One (Laplace) smoothing formula for a bigram model.

bigram 모델에서 Add-One smoothing 식을 쓰라.

(3) Given the corpus:

<s> i like nlp </s>
 <s> i like math </s>
 <s> i write code </s>

Vocabulary size is $|V| = 8$ excluding <s> but including </s>.

Compute $P(P_{\text{Laplace}}(\text{code} \mid \text{like}))$.

위 corpus에서 $|V|=8$ 일 때 $P(P_{\text{Laplace}}(\text{code} \mid \text{like}))$ 를 계산하라.

(4) In Naive Bayes text classification, why do we compute $P(c) \prod_i P(w_{i|c})$ instead of directly estimating $P(c|d)$?

Naive Bayes 문서분류에서 왜 $P(c|d)$ 를 직접 추정하지 않고 $P(c) \prod_i P(w_{i|c})$ 를 계산하는가?

answer

(1) N-gram sentence probability는 조건부 확률들의 곱이다.

$$P(w_1, \dots, w_T) = \prod_t P(w_t \mid w_{t-n+1:t-1})$$

이 중 하나라도 count가 0이면 해당 조건부 확률이 0이 되고, 곱 전체가 0이 된다.

(2)

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

분자에 1을 더해 unseen bigram도 0이 되지 않게 하고, 분모에 vocabulary 크기를 더해 전체 확률합을 보장한다.

(3) Corpus에서 like는 두 번 등장한다.

- $C(\text{like}, \text{code}) = 0$
- $C(\text{like}) = 2$
- $|V| = 8$

$P_{\text{Laplace}}(\text{code} \mid \text{like}) = \frac{0+1}{2+8} = \frac{1}{10} = 0.1$

(4) 문서 d 전체에 대해 $P(c|d)$ 를 직접 추정하려면 가능한 문서 조합이 너무 많아 data sparsity가 심하다. Bayes rule을 이용해

$$P(c|d) \propto P(c) \prod_i P(w_i|c)$$

로 바꾸고, Naive Bayes의 조건부 독립 가정으로

$$P(d|c) = \prod_i P(w_i|c)$$

를 사용한다. 즉, 어려운 “문서 $\rightarrow$ 클래스” 확률을, 더 추정하기 쉬운 “클래스별 단어 출현 확률”로 proxy한다.

예상 3. Relation Extraction, GraphRAG, Knowledge Graph Embedding

Answer the following questions about relation extraction and knowledge graphs.

(1) State three approaches to Relation Extraction discussed in the wrap-up.

랩업에서 다룬 Relation Extraction의 세 가지 접근법을 쓰라.

(2) Explain Distant Supervision and Snowballing.

Distant Supervision과 Snowballing을 설명하라.

(3) List the pipeline from raw text to Knowledge Graph.

raw text에서 Knowledge Graph까지 가는 파이프라인을 나열하라.

(4) What type of question is GraphRAG better suited for than plain Vector RAG?

GraphRAG가 단순 Vector RAG보다 더 잘 다루는 질문 유형은 무엇인가?

(5) In a TransE-style knowledge graph embedding model, what relation should hold among the head, relation, and tail vectors? Write the margin loss.

TransE류 KG embedding에서 head, relation, tail 벡터 사이에 어떤 관계가 성립해야 하는가? margin loss를 쓰라.

answer

(1) 세 접근법

1. Rule-based: Hearst Pattern, regular expression 등을 사용.
2. Supervised: feature를 만들고 classifier를 학습. 예: Logistic Regression.
3. Distant Supervision: 외부 KB/사전에 있는 relation을 텍스트에 자동 라벨링해 학습 데이터로 사용.

(2)

- Distant Supervision: KB에 (Barack Obama, bornIn, Hawaii)가 있고 한 문장에 Barack Obama와 Hawaii가 함께 등장하면, 그 문장을 bornIn relation의 positive example처럼 자동 라벨링하는 방식.
- Snowballing: 초기 사전으로 relation을 추출하고, 추출된 triple로 사전을 키운 뒤, 더 많은 relation을 다시 추출하는 bootstrap 방식.

(3) 파이프라인

Tokenization → POS Tagging → NER → Entity 인식/정규화 → Relation Extraction → Triple (head, relation, tail) 생성 → Knowledge Graph 구성.

(4) GraphRAG는 multi-hop reasoning이나 aggregation 질문에 강하다.

예:

- “A의 회사의 CEO의 출신 대학은?”
- “어떤 질병과 연결된 약물 중 가장 많이 등장하는 것은?”

단순 Vector RAG는 비슷한 문서를 찾는 데 강하지만, relation을 따라 여러 hop을 이동하거나 graph 구조에서 집계하는 질문에는 약하다.

(5) TransE의 핵심 제약:

$$\|h+r-\text{approx } t\|$$

positive triple은 $\|h+r\|$ 이 $\|t\|$ 에 가까워야 하고, corrupted negative triple은 멀어야 한다.

Margin loss:

$$\mathcal{L} = \sum_{(h,r,t) \in S} \sum_{(h',r,t') \in S'} \left[\gamma + d(h+r, t) - d(h'+r, t') \right]_+$$

- S : positive triple set
- S' : negative/corrupted triple set
- d : L1 또는 L2 distance
- γ : margin
- $[x]_+ = \max(0, x)$

예상 4. Cross-lingual NLP: Orthogonal Procrustes and Isomorphism

(1) Define the isomorphism assumption in cross-lingual word embedding alignment.

Cross-lingual word embedding alignment에서 isomorphism 가정을 정의하라.

(2) What is Orthogonal Procrustes? What type of transformation does it learn?

Orthogonal Procrustes는 무엇이며 어떤 변환을 학습하는가?

(3) Why does an orthogonal transformation preserve similarities within a monolingual embedding space?

직교 변환이 단일 언어 임베딩 공간 내부의 유사도를 보존하는 이유는 무엇인가?

(4) Explain how multilingual LLMs can be understood as implicitly performing a similar alignment.

다국어 LLM이 명시적 Procrustes 없이도 유사한 정렬을 암묵적으로 수행한다고 볼 수 있는 이유를 설명하라.

answer

(1) Isomorphism 가정은 서로 다른 언어의 embedding space가 완전히 같지는 않더라도, 의미 관계의 구조는 유사하다는 가정이다. 예를 들어 어떤 언어에서는 “왕”은 “여왕”과 가깝고 “책”보다는 멀다. 즉 단어들 사이의 상대적 관계 구조가 언어 간에 보존된다고 본다.

(2) Orthogonal Procrustes는 source language embedding을 target language embedding에 맞추기 위해 최적의 orthogonal matrix W 를 찾는 문제이다.

$$W^* = \arg\min_W \|XW - Y\|_F$$

여기서 W 는 회전(rotation)과 반사/reflection 같은 직교 변환을 나타낸다.

(3) 직교 행렬은 내적과 norm을 보존한다.

$$\|xW\| = \|x\|$$

또한 두 벡터의 각도와 cosine similarity도 보존된다. 따라서 source language 내부의 단어 간 유사도 구조를 망가뜨리지 않고 target language 쪽으로 공간을 정렬할 수 있다.

(4) 다국어 LLM은 여러 언어의 텍스트를 같은 모델 파라미터 공간에서 함께 학습한다. 이 과정에서 의미가 비슷한 표현들이 언어와 무관하게 비슷한 representation을 갖도록 압력이 생긴다. 명시적으로 Procrustes matrix를 학습하지는 않지만, 내부 representation이 언어 간 의미 정렬을 암묵적으로 수행한다고 볼 수 있다.

예상 5. Code Evaluation: CodeBLEU and Pass@K

(1) Why are token-overlap metrics such as BLEU insufficient for code generation evaluation?

BLEU 같은 token-overlap metric이 code generation 평가에 부족한 이유를 설명하라.

(2) What additional information does CodeBLEU consider compared to BLEU?

CodeBLEU는 BLEU에 비해 어떤 정보를 추가로 고려하는가?

(3) Define Pass@K conceptually.

Pass@K의 개념적 정의를 쓰라.

(4) Write the unbiased estimator for Pass@K when n samples are generated and c of them are correct.

n개 샘플 중 c개가 정답일 때 Pass@K의 unbiased estimator 식을 쓰라.

(5) Numerical exercise. For one problem, a model generates n=20 programs, and c=3 of them pass all unit tests. Compute pass@1 and pass@5.

한 문제에서 모델이 n=20개 코드를 생성했고, 그중 c=3개가 unit test를 통과했다. pass@1과 pass@5를 계산하라.

answer

(1) 코드는 token overlap이 높아도 틀릴 수 있고, token overlap이 낮아도 의미적으로 같은 코드일 수 있다. 변수명 변경, 순서 변경, 동치 구현은 BLEU를 낮출 수 있지만 실행 결과는 같을 수 있다. 반대로 ==와 =처럼 작은 token 차이가 semantic error를 만들 수도 있다.

(2) CodeBLEU는 BLEU의 n-gram overlap에 더해 syntax tree, data-flow 같은 code-specific 구조 정보를 고려한다. 즉 코드의 구문적/의미적 유사도를 더 반영하려는 metric이다.

(3) Pass@K는 한 문제에 대해 K개의 코드를 생성했을 때, 그중 하나라도 unit test를 통과하는 정답 코드가 있을 확률이다. execution 기반 metric이다.

(4)

$$\text{pass@k} = \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

이는 k개를 뽑았을 때 모두 오답일 확률의 여집합이다.

(5)

n=20, c=3, 따라서 오답 수는 n-c=17.

pass@1:

$$1 - \frac{\binom{17}{1}}{\binom{20}{1}} = 1 - \frac{17}{20} = \frac{3}{20} = 0.15$$

pass@5:

$$1 - \frac{\binom{17}{5}}{\binom{20}{5}}$$
$$\binom{17}{5} = 6188, \quad \binom{20}{5} = 15504$$

$$\text{pass@5} = 1 - \frac{6188}{15504} \approx 1 - 0.3991 = 0.6009$$

따라서 pass@5는 약 0.601이다.

예상 6. PEFT, GRPO, DPR/ColBERT 종합 변형

The following questions ask you to compare several modern NLP/LLM techniques from the wrap-up.

(1) Explain the motivation of PEFT. How are Adapter and LoRA different from full fine-tuning?

PEFT의 목적을 설명하라. Adapter와 LoRA는 full fine-tuning과 어떻게 다른가?

(2) What is the key idea of LoRA as a low-rank update?

LoRA의 low-rank update 핵심 아이디어를 설명하라.

(3) In GRPO, what is removed compared to PPO, and how is the advantage computed instead?

GRPO는 PPO와 비교해 무엇을 제거하며, advantage를 대신 어떻게 계산하는가?

(4) In DPR, explain the role of in-batch negatives.

DPR에서 in-batch negative의 역할을 설명하라.

(5) Compare Bi-encoder, Cross-encoder, and ColBERT in terms of representation and computational cost.

Bi-encoder, Cross-encoder, ColBERT를 representation 방식과 계산 비용 측면에서 비교하라.

answer

(1) PEFT(Parameter-Efficient Fine-Tuning)의 목적은 대규모 모델 전체를 업데이트하지 않고, 일부 작은 파라미터만 학습해 비용을 줄이는 것이다.

- Full fine-tuning: 기존 모델의 모든 weight를 업데이트.
- Adapter: 기존 모델은 대부분 freeze하고 작은 adapter module을 삽입해 그 부분만 학습.
- LoRA: 기존 weight를 직접 전부 바꾸지 않고 low-rank update matrix만 학습.

PEFT는 catastrophic forgetting 방지가 주목적인 continual learning과 다르다. 랩업에서는 주 목적을 돈/계산 비용 절약으로 강조했다.

(2) LoRA는 weight update ΔW 를 full-rank matrix로 직접 학습하지 않고 두 작은 행렬의 곱으로 표현한다.

$\Delta W = BA$

여기서 rank r 이 작으면 (A, B) 의 파라미터 수가 원래 (W) 보다 훨씬 적다. 기존 weight (W) 는 freeze하고 low-rank update만 학습하므로 memory와 computation이 줄어든다.

(3) GRPO는 PPO에서 필요한 value model / critic을 제거한다.

PPO는 advantage를 $(Q-V)$ 또는 value model 기반으로 계산하지만, GRPO는 같은 prompt에 대해 여러 답변을 생성해 group을 만들고, 각 답변 reward를 group 평균/표준편차로 normalize해 relative advantage를 만든다.

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

즉 절대평가 value model 대신 group 내부 상대평가를 사용한다.

(4) DPR은 query encoder와 passage encoder를 따로 둔 bi-encoder 구조이다. 관련 query-passage pair의 dot product는 높이고, 관련 없는 pair의 dot product는 낮추도록 학습한다.

In-batch negative는 한 mini-batch 안에서 다른 query의 positive passage를 현재 query의 negative passage로 간주하는 방법이다.

장점:

- 별도로 negative를 찾는 비용이 줄어든다.
- 같은 batch 안의 positive passage는 어느 정도 그럴듯한 문서이므로 hard negative 역할을 할 수 있다.
- “공짜 hard negative”처럼 작동한다.

(5)

| 모델 | Representation | 계산 비용 | 장단점 |

|---|---|---|---|

| Bi-encoder | query 벡터 1개, document 벡터 1개 | 낮음 | 미리 document embedding 가능. 대규모 검색에 적합. query-document 상호작용은 약함 |

| Cross-encoder | query와 document를 함께 입력 | 매우 높음 | 모든 token이 서로 attention 가능해 성능 좋음. 수백만 문서 전체 검색에는 부적합 |

| ColBERT | query token별 vector 다수, document token별 vector 다수 | 중간 | token-level similarity를 계산해 bi보다 표현력 좋고 cross보다 가벼움. Multi-vector dense retrieval |

보통 실무에서는 bi-encoder로 100~200개 후보를 빠르게 가져오고, cross-encoder로 re-ranking한다. ColBERT는 bi와 cross의 중간 스펙트럼에 있다.